

DPI343

Computer Organization and Assembly Language

Lecture 13

Principles of video output

Lectures - prof. em. U. Sukovskis

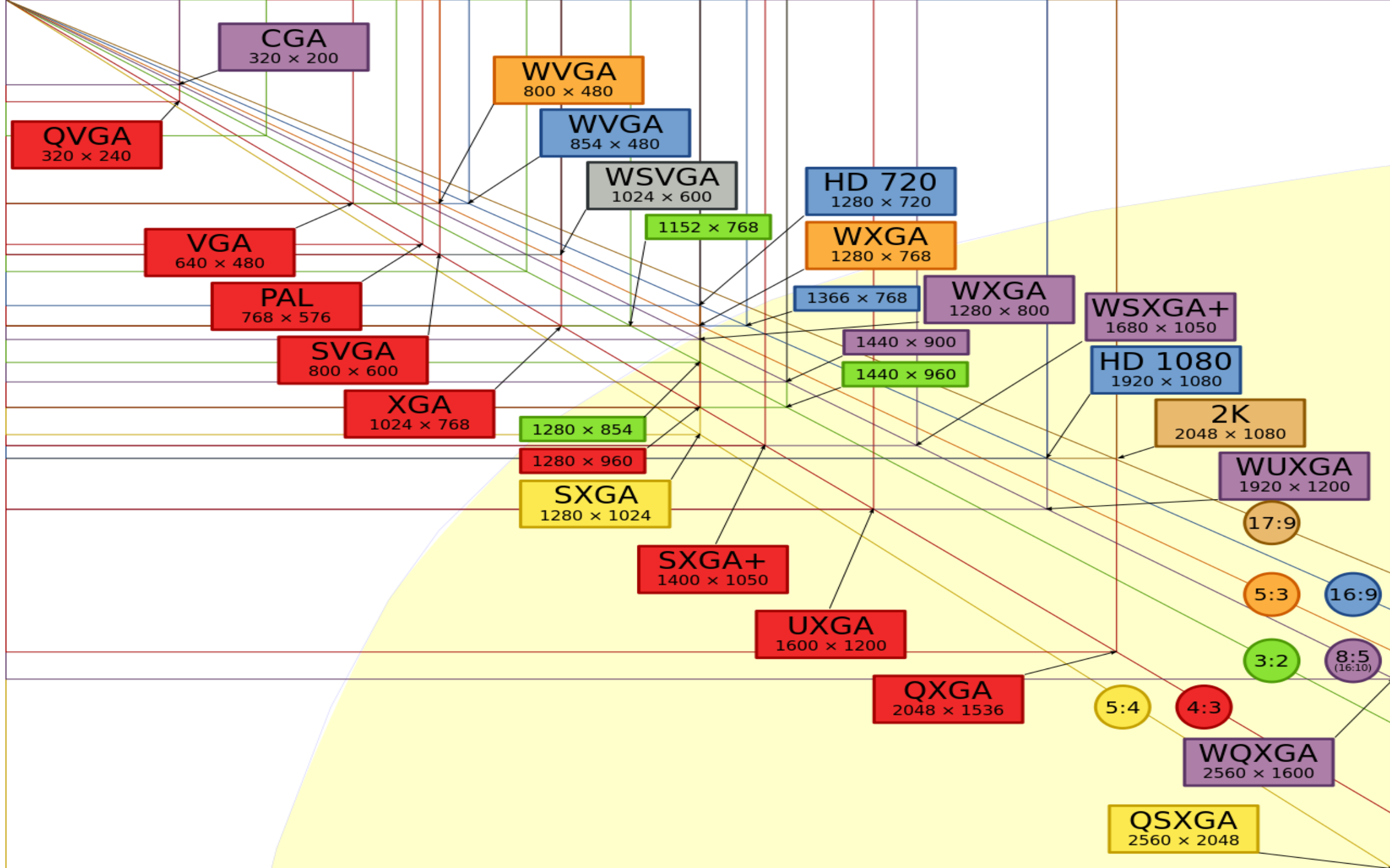
Labs for local students - assoc. prof. P. Rusakovs

Labs for foreign students - assoc. prof. G. Alksnis

Autumn 2025



RTU
DATORZINĀTNES,
INFORMĀCIJAS
TEHNOLOĢIJAS
UN ENERĢĒTIKAS
FAKULTĀTE



BIOS video interface

- Different video adapter cards and monitors are available
- Each adapter supports several different **display modes**
- There are several **text modes** and several **graphic modes**
- Video adapter enables screen image creation using data stored in the **video memory**
- For VGA type adapters
 - **Text mode** video memory in Real address mode is mapped to address **B800:0000**
 - **Graphic mode** video memory in Real address mode is mapped to address **A000:0000**
- BIOS interrupt **int 10h** can be used to set video mode and perform other operations

BIOS video modes

- Text modes: 0, 1, 2, 3, 7
 - e.g. mode 3 - 25 rows, 80 columns, 16 colours
- Graphic modes: 4, 5, 6, 8, 9, 10, ...
 - e.g. mode 12h - resolution 480x640, 16 colours
- Current video mode number is stored in the BIOS Data Area at address 0000:0449 (1 byte)

- BIOS interrupt to set video mode:

```
mov     ah, 0           ; BIOS function 0 - Set video mode
mov     al, 12h         ; al = video mode
int     10h
```

- BIOS interrupt to get current video mode:

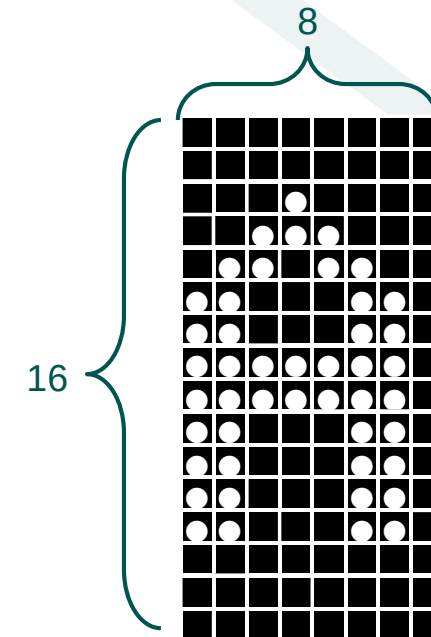
```
mov     ah, 0Fh         ; BIOS function 0Fh - get video mode
int     10h             ; returns current video mode in al
:::
```

Text mode

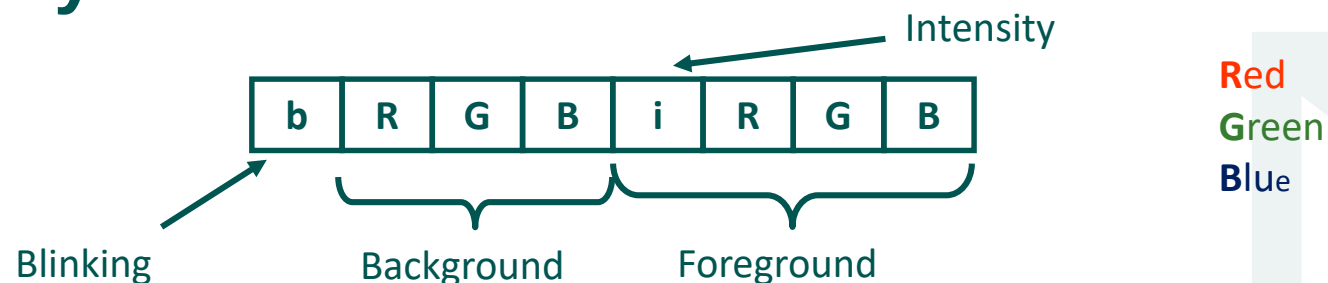
- For VGA type adapters text mode video memory address is mapped to B800:0000
- **Two memory bytes** are allocated for each symbol to display on screen



- Adapter uses **ASCII code** to find and display **symbol bitmap** picture using **foreground and background** colors defined in the **attribute** byte
- For example, ASCII code of capital letter A is 41h and default bit pattern is used to draw the picture of the letter
- Default patterns of pixels are used by the video adapter for all symbols. These patterns can be replaced using BIOS interrupt int 10h



Text attribute byte



Example:

attribute byte value 1Eh = 00101110b gives yellow symbol on a green background

Standard colors

0000	0	Black	1000	8	Grey
0001	1	Blue	1001	9	Bright blue
0010	2	Green	1010	10	Bright green
0011	3	Cyan	1011	11	Bright cyan
0100	4	Red	1100	12	Bright red
0101	5	Magenta	1101	13	Bright magenta
0110	6	Brown	1110	14	Yellow
0111	7	White	1111	15	Bright white

Number of the color actually is the number of the palette register used by the video adapter to display the color

Direct text output

To display one symbol on screen in the text video mode 3
(25 rows, 80 columns, 16 colours):

- store ASCII code in the video memory at address **B800:offset**, where
 $offset = 2 * (80 * (row - 1) + (column - 1))$
- store attribute byte in the video memory at address **B800:offset+1**

Examples:

- the offset value of the first symbol in the first row is 0,
- the offset value of the second symbol in the third row is
 $2 * (80 * (3 - 1) + (2 - 1)) = 322$

Task

Write text OK in the center of the text screen using red letters and white background

Solution

```
mov  ax, 13      ; row
mov  bx, 39      ; col
; Calculate offset in the video memory
dec  ax          ; ax = row-1
mov  dl, 80      ; dl = 80
imul dl          ; ax = (row-1)*80
dec  bx          ; bx = col-1
add  ax, bx      ; ax = (row-1)*80 + (col-1)
add  ax, ax      ; ax = ax * 2
; Calculated offset value is now in the register AX

; Set video memory starting address in ES:DI
mov  bx, 0B800h
mov  es, bx      ; es = segment value
mov  di, ax      ; di = offset value

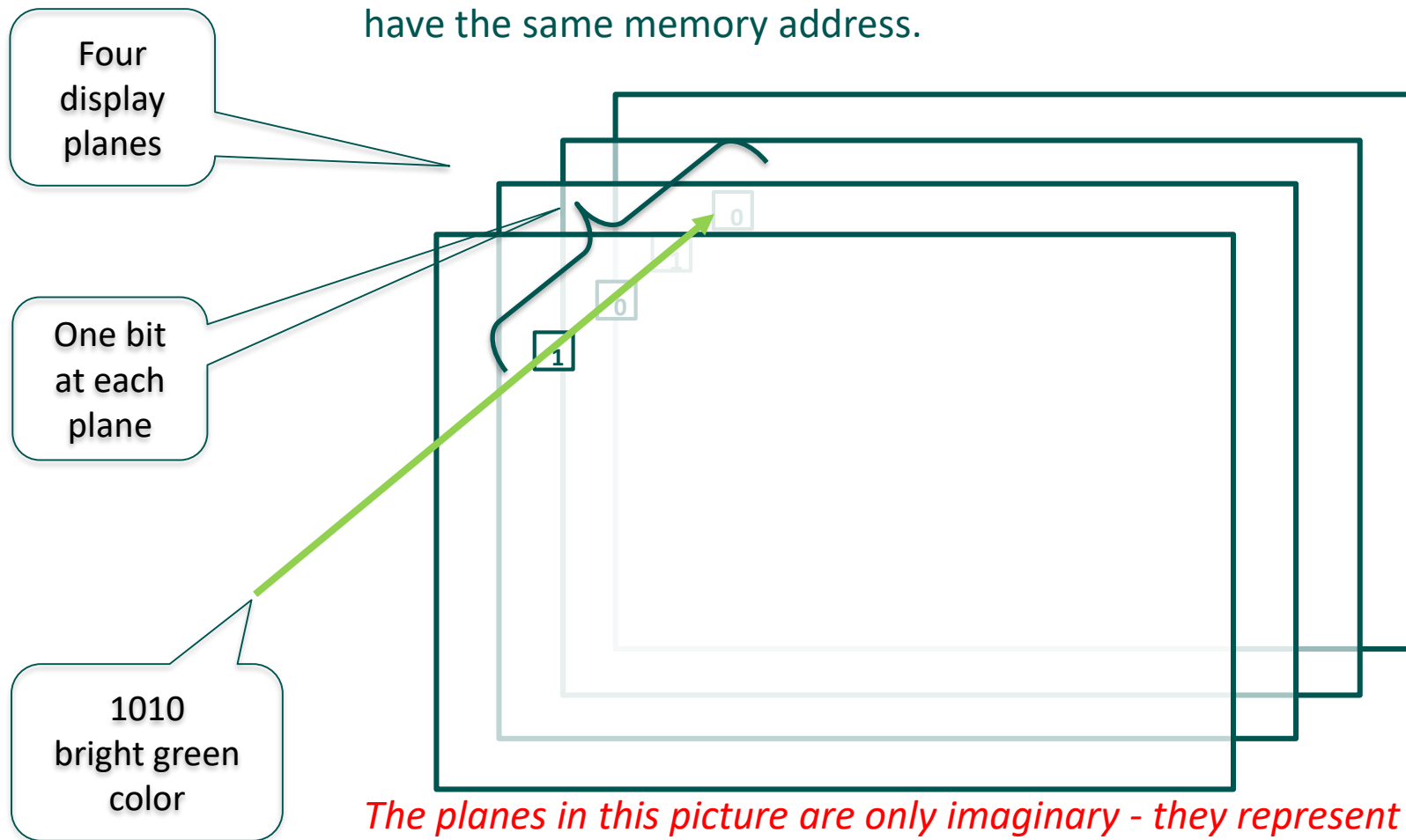
; Write symbols and attributes
mov  byte ptr es:[di], '0'
mov  byte ptr es:[di+1], 01110100b
mov  byte ptr es:[di+2], 'K'
mov  byte ptr es:[di+3], 01110100b
```

VGA graphic mode

- For VGA graphic mode video memory address is mapped to **A000:0000**
- **Each pixel on the screen corresponds to one bit in all display planes** of the video memory. Combination of bits from all planes determines the color of the pixel
- High order bit of the byte A000:0000 corresponds to the pixel in the upper left corner of the screen
- **Each display plane memory has the same starting address A000:0000**
- Write and read operations are performed through Latch registers.
- Different registers of the video adapter (bit mask register, planes mask register, etc.) are used to control operations
- Ports are used to access registers of the video adapter
- The VGA hardware has 2 read modes and 4 write modes.
- **The VGA hardware can perform bit manipulation on data and allow to operate on all display planes in a single operation**

Example: 16 color mode with 4 display planes

Color of the pixel is defined by the combination of bits at the same position in all display planes. All 4 bits are in the bytes of different plane but all 4 bytes have the same memory address.



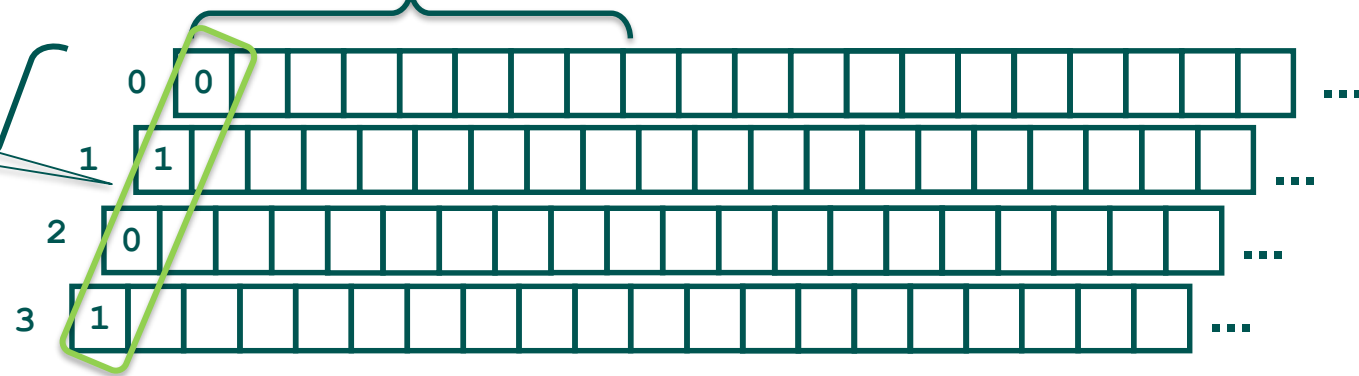
The planes in this picture are only imaginary - they represent the data structure in memory, allow us to understand the principle of operation, and are not physical overlays on the screen surface.

Writing a data byte on screen in VGA 16 color graphics mode (write mode 2)

Bright green pixel in the upper left corner of the screen

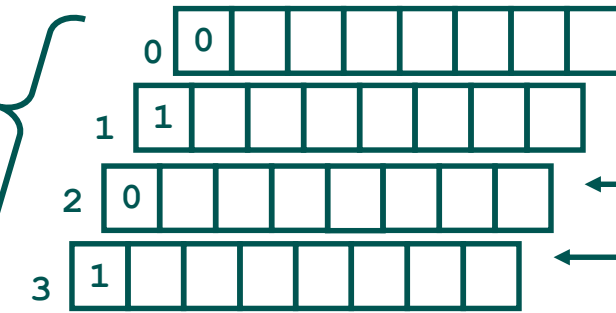
4 display planes
A000:0000

First byte



Latch registers

Data byte will only be copied to one of the bits of latch registers – the high order bit



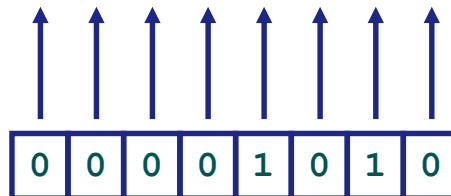
Bit Mask



Planes Mask

Data byte will be copied to each latch register, i.e. it goes to each plane

Data byte contains the color code 1010 (bright green)



Assembly code:

```
mov ax, 0A000h ; Segment value  
mov es, ax ; in ES register  
mov es: [0000h], bl ; Copy data byte  
; from BL to  
; video memory
```

Writing a data byte on screen in VGA 16 color graphic mode (*write mode 2*)

- **Write mode 2** is used to write a pixel color value from the lower 4 bits of the data byte into 4 display planes. The bit representing each plane will be replicated across all 8 bits of the corresponding planes.
- The **Bit Mask** register is used to select bits from the data byte. The resulting data is written to the latch registers of display planes enabled in the **Planes Mask**.
- **Latch registers** are copied to memory bytes of corresponding display planes.
- To set color of just one pixel you can use a bit mask with only one of the bits with a value 1. In this case, data byte bits will be copied to only one bit in each of the latch registers.

How to set the controller registry values

- *Write mode can be set using register 5 of the graphics controller (GDC)*
- *Number of the GDC register can be set via port 3CE, data via port 3CF*
- *Value of the Bit Mask can be set using GDC register 8*
- *Value of the Planes Mask can be set using register 2 of the Sequencer*
- *Number of the Sequencer register can be set via port 3C4, and data via port 3C5*

Example

Write one pixel on screen in VGA graphics mode with resolution 640x480 and 16 colors using procedure with 3 parameters: coordinates X, Y, and color.

Coordinates of the upper left corner are $X=0$, $Y=0$.

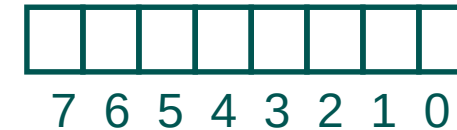
1) Calculate offset of the byte in the video memory:

$$\text{offset} = 80 * Y + X / 8$$

*Why $80*Y$? One row of 640 pixels is represented by 80 bytes in each of the 4 display planes (a total of $80*4 = 320$ bytes of video memory are used for one row of pixels).*

2) Calculate number of the bit inside this byte:

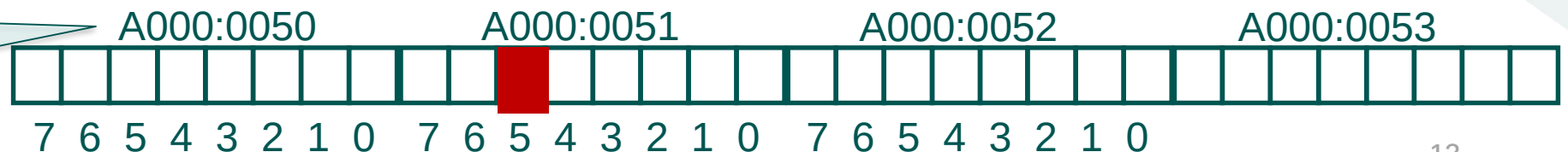
$$\text{bitnum} = 7 - X \% 8$$



EXAMPLE: $X=10$ and $Y=1$ (i.e. 11th pixel in the 2nd row) $\text{offset} = 80 * 1 + 10 / 8 = 81$,
number of the bit in this byte is $\text{bitnum} = 7 - 10 \% 8 = 7 - 2 = 5$

640 pixels per row,
one byte for 8
pixels, i.e. 80 bytes
for one row, so the
2nd row starts at
offset 80 = 50h)

2nd row



Example (cont.)

```
putpx    proc far
; Procedure to write one pixel on 640x480 VGA 16 color screen. Parameters: X, Y, COLOR
        push    bp                ; save registers
        mov     bp,sp
        push    ax
        push    bx
        push    cx
        push    dx
        push    es
x        equ     [bp+10]          ; parameter x (defined name)
y        equ     [bp+8]           ; parameter y (defined name)
color    equ     [bp+6]           ; parameter color (defined name)
        ; Calculate offset of the byte
        mov     ax, y             ; coordinate y
        mov     dx, 80            ; 640 pixels per row / 8 pixels per byte = 80
        imul    dx                ; 80*y
        mov     bx, x             ; coordinate x
        mov     cl, 3             ;
        shr     bx, cl            ; x/8
        add     bx, ax            ; bx = offset = 80*y + x/8
        ; prepare bit mask
        mov     ax, x             ; ax=coordinate x
        mov     dl, 8
        idiv    dl                ; x/8 ah = remainder
        mov     cl, ah            ; cl = position of the bit in the data byte for this pixel
        mov     ah, 10000000b     ; mask
        shr     ah, cl            ; shift 1 to the corresponding bit (AH=bit mask value)
```

Example (cont.)

```
; set write mode 2
    mov     dx, 3CEh    ; port 3CEh to set GDC register number
    mov     al, 5       ; register 5
    out     dx, al      ; set GDC register number
    inc     dx          ; port 3CF to set value
    mov     al, 2       ; write mode 2
    out     dx, al      ; set value of the GDC register 5

; set Bit Mask
    mov     dx, 3CEh    ; port 3CEh to set GDC register number
    mov     al, 8       ; register 8 (Bit Mask)
    out     dx, al      ; set GDC register number
    inc     dx          ; port 3CF to set value
    mov     al, ah      ; copy bit mask value from AH to AL
    out     dx, al      ; set value of the Bit Mask register

; set Planes Mask
    mov     dx, 3C4h    ; port 3C4h to set Sequencer register number
    mov     al, 2       ; register 2 (Planes Mask)
    out     dx, al      ; set Sequencer register number
    inc     dx          ; port 3C5 to set value
    mov     al, 00001111b ; value of the planes mask (all 4 planes)
    out     dx, al      ; set value of the Sequencer register 2
```

Example (cont.)

```
; write a data byte
mov ax, 0A000h
mov es, ax                ; es = segment of the video memory
mov     al, es:[bx]       ; read one byte from the video memory to fill Latch registers
mov     ax, color         ; al = data byte (4 low order bits - one bit per one of the 4 planes)
mov     es:[bx], al       ; write the data byte according to Bit Mask and Planes Mask
; restore registers and return
pop     es
pop     dx
pop     cx
pop     bx
pop     ax
pop     bp
ret     6                  ; adjust top of the stack (3 parameters = 6 bytes)
putpx   endp
```

To call the procedure:

```
push    10                ; parameter x
push    1                 ; parameter y
push    15                ; parameter color
call    putpx
```